

Actividad 10 — Batalla naval en red (sockets)

Duración: 50 min · **Modalidad:** individual (en parejas para probar) · **Entrega:** repo GitHub público

1. Objetivos

- Entender qué es una **red** y los **sockets** en Python.
- Programar un **servidor** que espera conexiones y un **cliente** que se conecta.
- Enviar y recibir datos (coordenadas de disparos) entre dos máquinas.
- Reutilizar la lógica de la actividad 8 y llevarla “a la red”.

2. Marco teórico

Una red de computadoras sirve para que programas de **distintas máquinas se comuniquen**. La herramienta base es el **socket**: un “tubo” por el que viajan datos.

El modelo **cliente-servidor** es el más simple:

Servidor (jugador 1)	Cliente (jugador 2)
socket()	socket()
bind(IP, puerto)	connect(IP_servidor, puerto)
listen()	
accept() <- espera al jugador 2 ----	
recv() / send() <----->	recv() / send()

- **IP** identifica una máquina en la red. 127.0.0.1 es “mi propia máquina”. Para jugar entre dos PCs de la misma red, el servidor escucha en 0.0.0.0 (todas sus interfaces) y el cliente se conecta a la IP del servidor (se ve con ipconfig en Windows o ip a en Linux).
- **Puerto**: número (ej. 9999) que identifica el servicio dentro de la máquina.
- **TCP**: protocolo que garantiza que los mensajes lleguen completos y en orden.
- send() y recv() envían y reciben **bytes**, por eso convertimos texto a bytes con .encode() y al revés con .decode().

3. Paso a paso

```
cd mis-videojuegos
mkdir clase_10_red
```

3.1 Servidor: clase_10_red/servidor.py

```
import socket
import random

def crear_tablero():
    return [ "." * 8 for _ in range(8)]
```

```

def colocar_barcos(t):
    for tam in (4, 3, 2):
        colocado = False
        while not colocado:
            horizontal = random.choice([True, False])
            if horizontal:
                x = random.randint(0, 7 - tam)
                y = random.randint(0, 7)
                celdas = [(x + i, y) for i in range(tam)]
            else:
                x = random.randint(0, 7)
                y = random.randint(0, 7 - tam)
                celdas = [(x, y + i) for i in range(tam)]
            if all(t[cx][cy] == "." for cx, cy in celdas):
                for cx, cy in celdas:
                    t[cx][cy] = "B"
                colocado = True

tablero = crear_tablero()
colocar_barcos(tablero)

# 1) Crear socket TCP
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# 2) Escuchar en todas las interfaces, puerto 9999
servidor.bind(("0.0.0.0", 9999))
# 3) Esperar un cliente
servidor.listen(1)
print("Servidor listo. Esperando jugador en el puerto 9999...")
cliente, direccion = servidor.accept()
print("Conectado:", direccion)

# 4) Ciclo de juego: recibir disparos y responder
while True:
    datos = cliente.recv(1024).decode()
    if not datos:
        break
    fila, col = map(int, datos.split(","))
    if tablero[fila][col] == "B":
        tablero[fila][col] = "X"
        respuesta = "TOCADO"
    else:
        respuesta = "AGUA"
    cliente.send(respuesta.encode())

cliente.close()
servidor.close()

```

3.2 Cliente: clase_10_red/cliente.py

```
import socket

# Conectate a 127.0.0.1 si el servidor está en tu PC,
# o a la IP del otro equipo (misma red).
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("127.0.0.1", 9999))

while True:
    fila = input("Fila (0-7): ")
    col = input("Columna (0-7): ")
    cliente.send(f"{fila},{col}".encode())
    print("Respuesta del servidor:", cliente.recv(1024).decode())

cliente.close()
```

4. Probalo

Opción A (misma máquina): abrí dos terminales.

```
# Terminal 1
python clase_10_red/servidor.py
```

```
# Terminal 2
python clase_10_red/cliente.py
```

Opción B (dos PC en la misma red): el servidor corre en una máquina y el cliente en otra, conectando a la IP del servidor:

```
ipconfig # en la máquina servidor, busca "Dirección IPv4"
```

Y en el cliente, cambiá ("127.0.0.1", 9999) por ("LA_IP_DEL_SERVIDOR", 9999).

Ojo: puede que el firewall pida permiso. Aceptá para redes privadas.

5. Actividad de cierre (para el commit)

1. **Victoria:** el servidor cuenta los tocados y, cuando se hundan todos los barcos (en este caso 3), envía "FIN:GANASTE" y cierra la conexión.
2. **Adivinanza del turno:** agregá al cliente que no pueda repetir una coordenada ya disparada.
3. **Chat:** aprovechá el mismo canal para enviar mensajes de texto además de disparos (por ejemplo el mensaje "FIN" para despedirse).
4. Commit y push:

```
git add .
git commit -m "Clase 10: batalla naval en red con sockets"
git push
```

6. Usando IA en esta clase (prompts sugeridos)

Prompt 1: “Explicame con una analogía qué es un socket, una IP, un puerto y la diferencia entre cliente y servidor.”

Prompt 2: “¿Por qué `recv(1024)` recibe un número máximo de bytes y qué pasa si mando un mensaje más largo?”

Prompt 3: “Mi cliente se conecta pero el servidor no responde. Este es mi código: [pegá ambos archivos]. ¿Qué puede fallar?”

Prompt 4: “Convertí este servidor para que dos clientes jueguen entre sí (mediador). Mostrame la estructura general.”

7. Desafíos para seguir practicando

- Armar la batalla naval **completa y gráfica** en red con Pygame (cliente con tableros y clicks).
- Un juego de **Pong en red** donde el servidor reenvía las posiciones de las paletas.
- Usar `threading` para que un servidor atienda **varios clientes** a la vez.

8. Rúbrica de evaluación

Criterio	Logrado	En proceso	No logrado
Servidor	Escucha y responde	Conecta pero mal	No corre
Cliente	Envía y recibe	Parcial	No corre
Comunicación en red	Funciona en 2 PC	Solo local	No
Integra la lógica del juego	Sí	Parcial	No